```
}

/**
 * 利用圆的八对称性绘制8个对称点
 * @param pixels 像素点集合
 * @param xc, yc 圆心坐标
 * @param x, y 当前点相对圆心的偏移
 */
void PlotCirclePoints(std::vector<D2D1_POINT_2F>& pixels,
                      int xc, int yc, int x, int y) {
    // 第一、二象限
    pixels.push_back(D2D1::Point2F(xc + x, yc + y));
    pixels.push_back(D2D1::Point2F(xc - x, yc + y));
    // 第三、四象限
    pixels.push_back(D2D1::Point2F(xc + x, yc - y));
    pixels.push_back(D2D1::Point2F(xc - x, yc - y));
    // 交换x和y，得到另外4个对称点
    pixels.push_back(D2D1::Point2F(xc + y, yc + x));
    pixels.push_back(D2D1::Point2F(xc - y, yc + x));
    pixels.push_back(D2D1::Point2F(xc + y, yc - x));
    pixels.push_back(D2D1::Point2F(xc - y, yc - x));
}
```

Listing 3: 中点画圆法核心代码

5.1.4 Bresenham 画圆法实现

```
/**
 * Bresenham画圆法
 * @param xc, yc 圆心坐标
 * @param r 半径
 * @return 圆上的像素点集合
 */
std::vector<D2D1_POINT_2F> BresenhamCircle(int xc, int yc, int r) {
    std::vector<D2D1_POINT_2F> pixels;
    int x = 0, y = r;

    // 决策参数初始值: d = 3 - 2r
    int d = 3 - 2 * r;

```